Apogee Object Storage Architecture

Denis Cheremisov

April 16, 2026

Abstract

This paper describes the architecture of an object storage system designed for scalability, high local and global performance, and superior diagnosability. These goals are achieved through a simplified component interaction model, where each component is strictly isolated within its own domain of responsibility.

Contents

1 Definitions **3**

2 Physical System Design **4**
2.1 Hot Storage Layer . . . 4
2.1.1 Maximum Performance Mode . . . 4
2.1.2 High Performance Mode . . . 5
2.1.3 Budget On-Premise Solution . . . 5
2.2 Cold Storage Layer . . . 5
2.3 Tuple Selection . . . 6
2.4 Metadata . . . 7
2.4.1 Records . . . 7
2.4.2 Fragments . . . 7
2.4.3 Links . . . 7
2.4.4 Containers . . . 8
2.4.5 Tuples . . . 8

3 Processes **9**
3.1 High-Level Overview . . . 9
3.1.1 Write Path . . . 9
3.1.2 Read Path . . . 10
3.2 Index Generation . . . 10
3.3 Deduplication . . . 10
3.4 Regular Data Upload Process . . . 11
3.5 Streaming Data Upload Process . . . 11
3.6 Read Process . . . 12
3.7 Migration from Heat to Cold . . . 12
3.8 Data Compaction and Garbage Collection (GC) . . . 13
3.8.1 Stripe Compaction . . . 13
3.8.2 Garbage Collection . . . 13
3.8.3 Compaction and GC Collisions . . . 15

4 System Invariants **17**
4.1 Referential Integrity Invariants . . . 17
4.2 Hot $\rightarrow$ Cold Migration Invariants . . . 18
4.3 WAL and Retry Invariants . . . 18
4.4 Deduplication and GC Invariants . . . 18
4.5 Execution Invariants . . . 19

5 Supplementary Materials **21**
5.1 Complete WritePath for multiple fragments . . . 21
5.2 Complete MultiWrite . . . 21
5.3 On WAL implementation using a message broker and KV storage . . . 22
5.3.1 Introduction . . . 22
5.3.2 Working with KV storage . . . 22
5.3.3 Working with the broker . . . 23

6 Specialized WAL Solution **23**
6.1 Consensus Node 24
6.2 Clients 24

7 Roadmap **26**
7.1 MVP 26
7.2 GC 26
7.3 Logic Evolution 26
7.4 Optimization 26
7.5 Description of the “Continuation” Mechanism 27

1 Definitions

The system is composed of the following components:

- **Stateless Coordinators.**
- **Cold Storage (Cold Tier)**, which consists of cold nodes:
  - A RAFT-based allocator for managing tuples and allocating *stripes* within them.
  - Disk tuples managed by the allocator.
- **Heat (Hot Tier)**, represented by hot nodes consisting of three machines in an identical configuration. Multiple hardware variants are possible depending on cost and performance requirements.
- **WAL (Write-Ahead Log)**: A distributed WAL. Currently, the simplest implementation uses a message broker and a KV store with persistence and consistency guarantees (e.g., NATS + JetStream, providing both broker and KV functionality).
- **Metadata**:
  - **Records**: Stores metadata for user records.
  - **Fragments**: Stores metadata for data fragments, including fields for deduplication support.
  - **Links**: A collection mapping Records to Fragments.
  - **Containers**: Describes the physical placement of fragment containers. A container can be either "sealed" within a cold tier stripe or temporarily reside in a hot tier buffer.
  - **Tuples**: Links cold tier tuples to physical disks.

2 Physical System Design

2.1 Hot Storage Layer

I propose three distinct approaches here, varying in hardware exoticism and cost. In all cases, the hot layer employs the maximum degree of redundancy. At this stage, it is most efficient to work from the "greatest common denominator"—specifically, using a $3\times$ replication scheme—and then reduce the redundancy level once the data moves to cold storage. Thus, when we refer to a "hot node", we mean a set of three physical nodes, each storing a replica of the data. Consequently, the corresponding drives on these machines must be equivalent in terms of hardware LBA and capacity.

In any mode, a disk D_i is represented as follows:

$$D_i = (B_{i1}, B_{i2}, \ldots, B_{im})$$
$$|B_{i1}| = |B_{i2}| = \cdots = |B_{im}| = 64\text{MiB}$$

The blocks B_{ij} are referred to as *buffers*.

2.1.1 Maximum Performance Mode

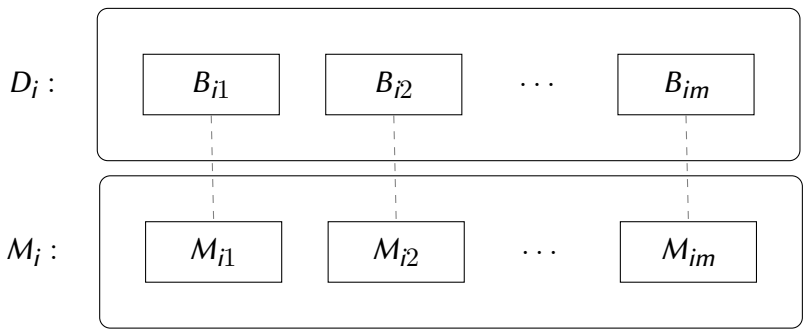
Hardware requirements:

- One or more NVMe SSDs.
- NVDIMM module(s) (offering similar guarantees to Optane but at a lower cost).

In this mode, we utilize NVMe SSDs—one, two, or three. Using more is rarely practical, as three drives will saturate any network, and a single NVMe on PCIe Gen5 can saturate it alone.

For each disk D_i, a memory region M_i is allocated on the NVDIMM modules. This region is likewise divided into equal fragments M_{ij}, where $j = 1, 2, \ldots, m$:

$$M_i = (M_{i1}, M_{i2}, \ldots, M_{im})$$
$$|M_{i1}| = |M_{i2}| = \cdots = |M_{im}|$$



The size of the fragments M_{ij} is determined by the physical block size of the disks. Each M_{ij} stores:

- Metadata for the container currently being filled (or already filled) in buffer B_{ij}.
- A buffer in the sense of buffered I/O for LBAs not yet filled during a write operation.

Subsequent writes will fill this buffer, after which it is flushed to the tail of B_{ij}.

Note the strategy for selecting write buffers: we move sequentially from the beginning of the disk to the end. This ensures that all B_{ij} blocks are filled nearly sequentially, providing uniform wear leveling for the storage medium. In the event of a failed write, no attempt should ever be made to rewrite to the same block. Such an attempt would constitute an overwrite, resulting in unnecessary disk wear. Instead, we simply skip the LBA range intended for that fragment. Fundamentally, we strictly adhere to an **append-only semantics**.

This approach enables the system to handle hundreds of high-definition video streams simultaneously.

2.1.2 High Performance Mode

This mode is similar to the previous one but operates without NVDIMM. In this scenario, buffer metadata is stored in an external database, but LBA-level packing is not performed. Specifically, if a write does not completely fill a storage block, the remainder of the block is zero-filled (padded with zeros), and the subsequent write begins at the start of the next block.

Overall, this approach provides high performance, though it is inferior to the NVDIMM-based variant. It is less suited for video streaming, as it would require handling LBA-level buffering on the coordinators. This is clearly suboptimal, as it would require the coordinators to have intimate knowledge of the "Hot Tier internals" (the "guts" of the Heat layer).

2.1.3 Budget On-Premise Solution

Disk as a circular buffer.

In this mode, we do not attempt any dense packing of buffers. If a written fragment exceeds the current buffer boundaries, the buffer is immediately sealed and prepared for migration to the Cold Tier, while the fragment is written to the next available buffer. This approach is ill-suited for streaming; thus, this type of Heat layer is only appropriate for document stores and similar workloads. However, it is highly cost-effective and can be implemented on any hardware, including HDDs.

2.2 Cold Storage Layer

The Cold Tier is divided into cold nodes, each consisting of a pair:

- A RAFT-based allocator.
- A set of disk tuples.

A tuple T is defined as a set of disks:

$$T = (D_1, D_2, \ldots, D_N)$$

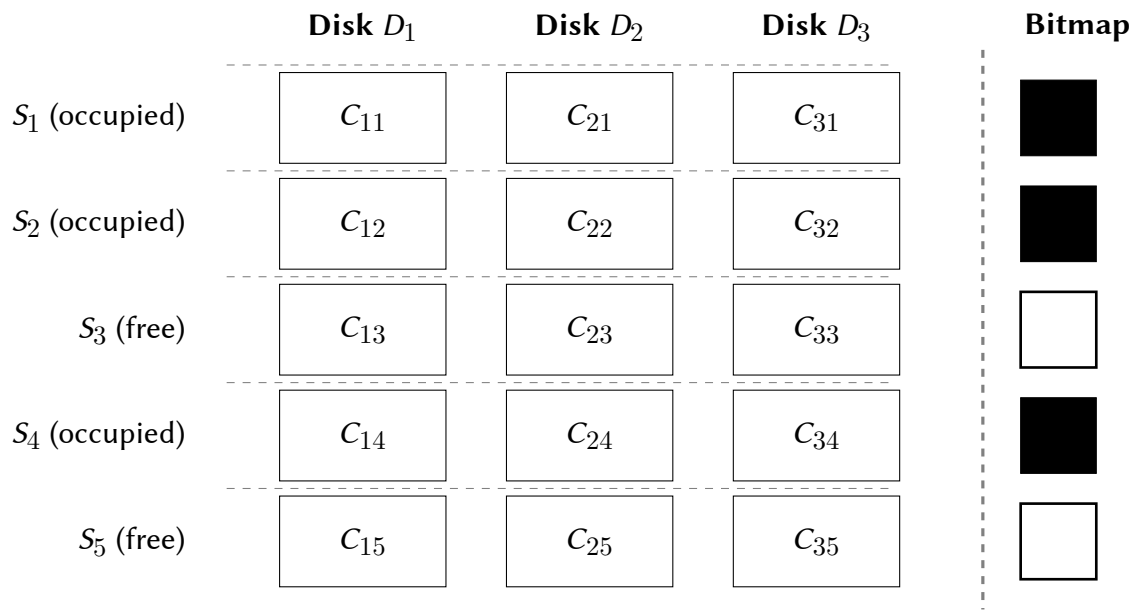
The disks themselves are represented as sets of blocks of uniform size:

$$D_i = (C_{i1}, C_{i2}, \ldots, C_{iM})$$

A *stripe* S_j within tuple T is defined as the set of blocks with index j collected from all disks in the tuple:

$$S_j = \left(C_{1j}, C_{2j}, \ldots, C_{Nj}\right)$$

The primary task of the RAFT allocator is to locate free stripes within the managed tuples. Specifically, it first selects a tuple and then searches for an available stripe. Tracking of free and occupied stripes is performed using a tuple *bitmap*, where each stripe is mapped to a bit: 1 for occupied and 0 for free.



Stripe allocation is performed in two steps:

1. Stripe **Lease**.
2. Confirmation of occupation (**Commit**) or cancellation (**Release**).

A *Release* is triggered automatically via TTL if no signal is received from the coordinator managing the migration within Δ seconds. Thus, moving 64 MiB of data requires only 2 RAFT operations. This scales to 64 RAFT operations per second for a aggregate data flow of 2 GiB/s. This overhead is negligible, especially considering that latency is not a primary concern for this background process.

2.3 Tuple Selection

When selecting a tuple, the goal is to favor those that are the least filled and not currently over-utilized. The following algorithm should be effective for this purpose.

Given a parameter N (a natural number), tuples are stored in the following structure:

```
tuples = RBTree[FreeStripes, LinkedList[Tuple]]
```

This means tuples are grouped by the number of available stripes they contain and are ordered by this count. Stripe selection for a write operation proceeds as follows:

1. Select a random K from the range $[0, N)$.
2. Retrieve the K-th tuple from the structure and initiate the data write process.
3. Upon confirmation of the write, remove the tuple from its current linked list and move it to the end of the list corresponding to the new (lower) free stripe count.

Highly loaded tuples can be skipped and ignored during the search for the K-th element. While this increases search complexity, the overhead remains minimal, as we are only performing approximately 32 such operations per 2 GiB/s of throughput.

2.4 Metadata

TiKV is the most suitable database for metadata storage, offering virtually unlimited horizontal scalability and high performance, particularly in `rawkv` mode.

2.4.1 Records

record.id	single	fragment.id	...

record.id	multi	no_of_fragments	...

This collection stores record metadata. For records consisting of a single fragment, we store the reference directly within the *Records* entry to avoid redundant storage. In scenarios involving small object storage, this optimization results in a $1.5\times$ reduction in the number of metadata entries by eliminating the need for *Links* records.

2.4.2 Fragments

fragment.id	container.id	offset	size	crc64	compression type	blake3	...

Stores fragment metadata.

2.4.3 Links

record.id	fragment.index	fragment.id

Stores mappings between records and fragments:

rid	0	fid_0
rid	1	fid_1
rid	2	fid_2
	...	
rid	n	fid_n

In practice, it is more efficient to store SPECK32(i) instead of sequential indices $0, 1, \ldots$. This prevents hotspots where a single shard receives a disproportionate number of entries from records with many fragments. By employing this deterministic permutation, entries are uniformly distributed across shards. While this sacrifices the ability to perform range scans for sequential fragment retrieval, this is not a significant drawback: since all fragments (except the last) are approximately 64 MiB, metadata retrieval speed will not become a bottleneck.

Consequently, the iteration logic changes from:

```
for x in records_fragments_iterator:
  read_fragment(x.fragment_id)
```

to:

```
for i in [0, record.no_of_fragments):
  fragment_id := records_fragments.get_fragment(record.id, speck32(i))
  read_fragment(fragment_id)
```

2.4.4 Containers

container.id	ref

Where `ref` can be either `Hot(hot.id, block.index)` or `Tuple(tuple.id, stripe.index)`.

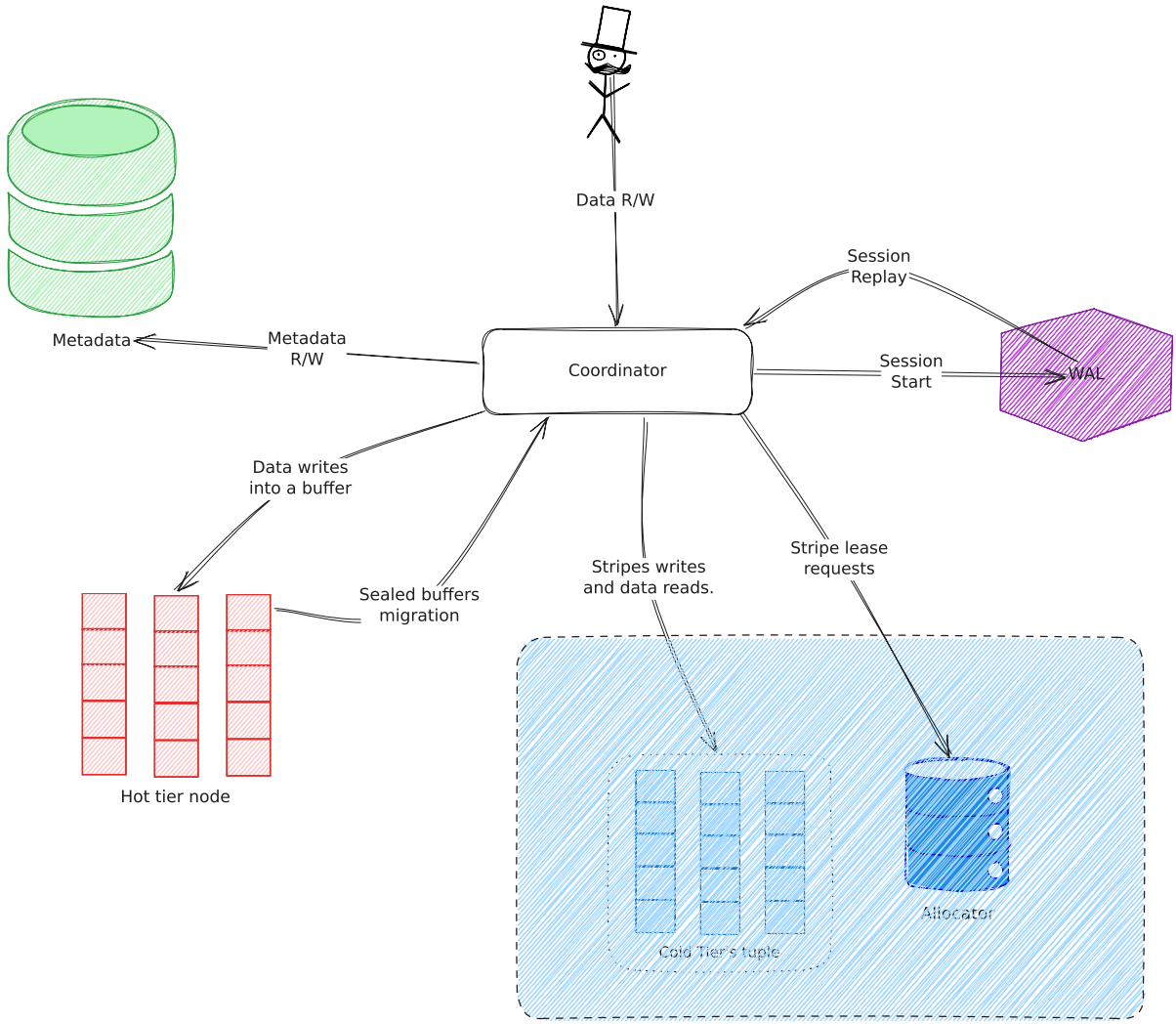
2.4.5 Tuples

tuple.id	payload

The `payload` is structured as follows:

```
{
    type: encoding scheme used (e.g., Replication or EC)
    disks: [
        (disk1, role1),
        ...
        (diskN, roleN),
    ]
}
```

3 Processes



3.1 High-Level Overview

Broadly speaking, the system's operational logic is as follows:

3.1.1 Write Path

1. The client **streams** data to the coordinator.
2. The coordinator **chunks** the stream into fragments, writes them to the *Heat* layer, and records the corresponding metadata in the metadata database.
3. The coordinator returns a *Record ID* to the client.

Writing to the *Heat* layer triggers periodic signals that initiate data migration from *Heat* to *Cold*:

1. The *Heat* layer notifies the **migrator** (which may reside within a coordinator) that a buffer is ready for transfer.

2. The migrator pulls the data from *Heat*, locates an available stripe in the *Cold* tier, writes the data, and subsequently updates the metadata database to reflect the new physical location.

3.1.2 Read Path

1. The client requests record data from the coordinator.
2. The coordinator queries the metadata database to locate the data and reads from the *Cold Tier* and/or *Hot Tier* accordingly.

3.2 Index Generation

An *etcd* instance is used to store `u128` counters for `record.id`, `fragment.id`, `container.id`, etc. Upon startup, a coordinator requests a range of IDs from *etcd* (e.g., 2^{20} values) and then allocates them locally one by one.

To ensure uniform distribution across shards, we use the `SPECK128(id)` encoded value rather than the raw identifier. This provides a high-quality uniform distribution. While an alternative would be adding a `hash(id)` column to the tables, this would incur additional storage overhead. `SPECK128` is a superior choice as it achieves the same goal without extra metadata.

3.3 Deduplication

Fragment data includes the following fields:

CRC64	Size	CompressionType	Blake3

We maintain a deduplication collection with the following schema:

CRC64	Size	CompressionType	Blake3	fragment.id	epoch

During a fragment write, we check this collection for existing matches. If a match is found, we evaluate its *epoch*:

- If the epoch is too far in the past, the existing fragment is ignored; we write a new fragment and update the deduplication record with the new reference.
- Otherwise, we skip the data write and reference the existing fragment.

This procedure may occasionally lead to minor data duplication due to race conditions (e.g., two concurrent processes writing the same fragment and updating the deduplication entry simultaneously). However, this is largely an *edge case*. In real-world scenarios, such occurrences should be extremely rare, and a scenario leading to significant storage bloat is purely theoretical.

3.4 Regular Data Upload Process

During a regular upload, where the user provides a dataset of a known size, the entire record R is partitioned into fragments:

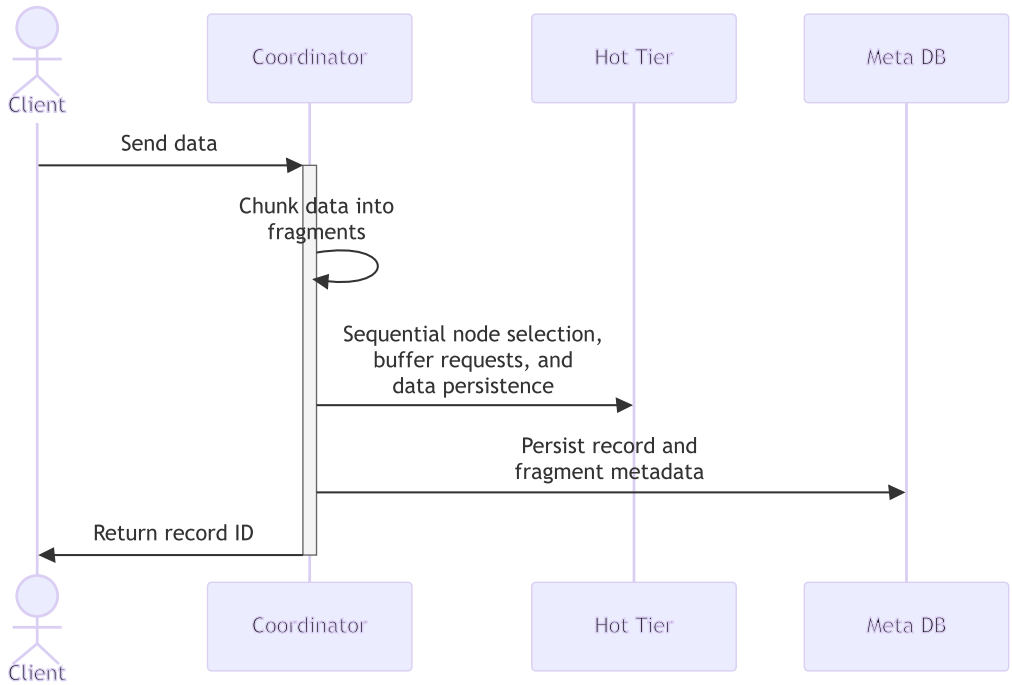
$$R = (F_1, F_2, \ldots, F_n)$$

where the lengths of all fragments are strictly identical, except for the last one, which may be smaller. Formally:

$$|F_1| = |F_2| = \cdots = |F_{n-1}| \geq |F_n|$$

This constraint is enforced to simplify the support for **range reads** (reads with an offset). While it is possible to split fragments into blocks of arbitrary sizes, doing so would significantly increase the implementation complexity and cost.

When a user submits data—simplified here, as the actual process involves WAL operations and a specific sequence of metadata database queries—the following occurs:



A complete, formal WritePath sequence, including commit and rollback support, is provided in the Appendices.

3.5 Streaming Data Upload Process

In this mode, we acquire temporary ownership of a buffer in the *Heat* layer with a confirmation. Data is then streamed into this buffer until it is full. Once filled, the buffer is **sealed**, and a task is queued for its migration to the *Cold Tier*.

A key distinction from the regular upload process is the fragment management: the fragment's status in the *Fragments* collection is marked as "in progress", and some of its metadata (such as current size) is stored within the buffer's own metadata. This optimization is necessary to avoid excessive write pressure on the metadata database during the streaming process.

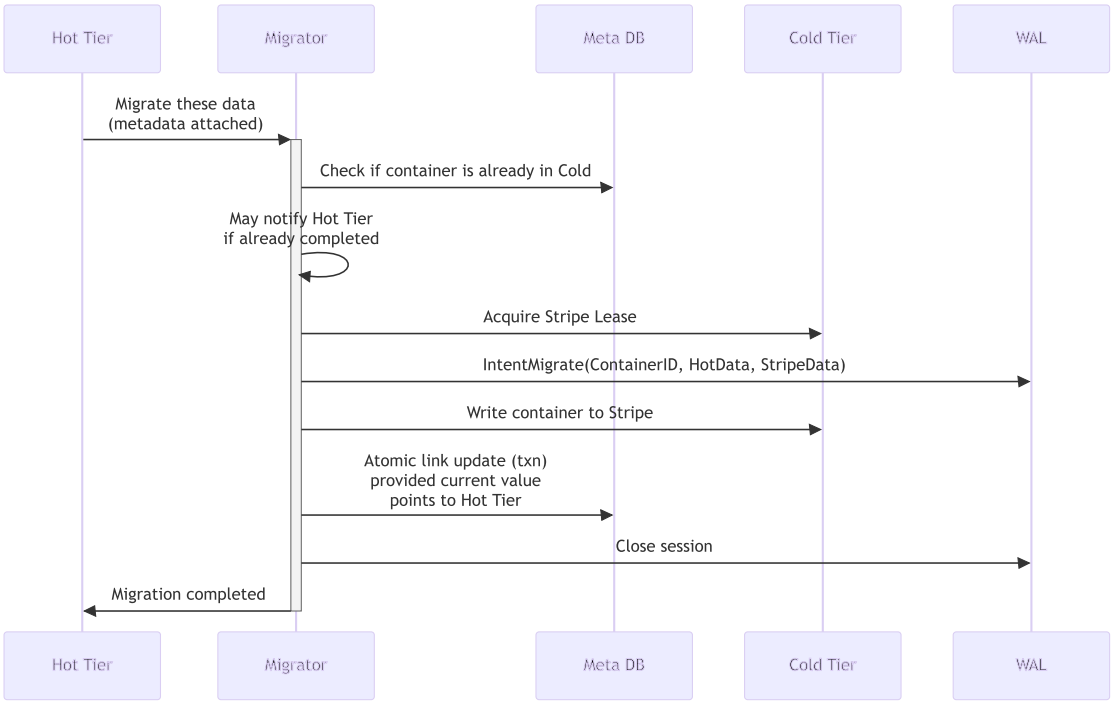
3.6 Read Process

The system supports both full reads and range reads (reads with an offset). The workflow is as follows:

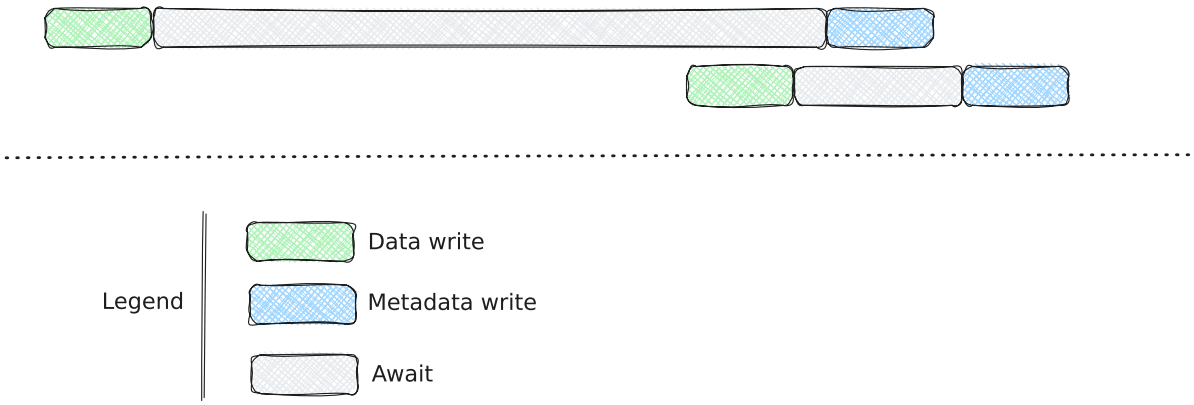
1. Retrieve the *Record* entry using the provided `RecordID`.
2. Retrieve fragment identifiers by scanning the *Links* collection using the key prefix `(RecordID, <fragment_index>)`.
3. For each fragment, fetch its metadata from the *Fragments* collection to identify the corresponding `ContainerID`.
4. Retrieve the container's metadata from *Containers* via the `ContainerID` to determine its physical placement (Heat or Cold tier).
5. If the container resides in the *Heat* layer, initiate a read request from the corresponding *Heat* buffer.
6. If the container is in the *Cold* tier (addressed via `TupleID` and `StripeIndex`), retrieve the tuple data from *Tuples* and calculate the required blocks based on the fragment's offset within the container.

3.7 Migration from Heat to Cold

When a buffer reaches a state where further filling is inefficient due to insufficient free space, the *Heat* layer initiates a migration procedure by invoking the **migrator**. This process follows the schema below:



The use of transactions is mandatory to prevent the creation of **orphaned resources**, as illustrated in the following collision scenario:



In this scenario, a previous migration process may experience significant latency (e.g., a "stale" or "zombie" process) and be considered timed out by the *Heat* layer, yet remain active and attempt a write after a second migration attempt has already started. Transactions are not prohibitive here: a single transaction corresponds to 64 MiB of data. For a system-wide write throughput of 2 GiB/s, this results in approximately 32 transactions per second—a negligible load for TiKV.

Furthermore, data persisted in the WAL ensures that a stripe acquired in the current session can be released if the buffer has already been successfully migrated by another process.

3.8 Data Compaction and Garbage Collection (GC)

3.8.1 Stripe Compaction

Stripe compaction involves reassembling N existing stripes into M new stripes, where $M < N$. Container metadata must track the following metrics: the amount of unoccupied space (e.g., failed fragment writes during filling or deleted fragments where space is considered free) and the average fragment size on disk.

A *naive compaction algorithm* functions as follows:

Iterate through the entries of the stripes being compacted and read the fragments into a memory buffer. During this process, if the next fragment exceeds the remaining buffer capacity (where the buffer size equals the Heat layer block size), a new stripe is allocated to store the accumulated data.

A *non-naive approach* is slightly more complex, involving fragment reordering and managing multiple buffers simultaneously to achieve optimal filling density for each.

Overall, this is a straightforward algorithmic task. Naturally, we log intents in the WAL during the process to track stripe states in case of a retry. Upon successful repacking (or immediately after a buffer is saved to a stripe), we perform a metadata update to point *Fragments* to the new locations.

3.8.2 Garbage Collection

This is perhaps the most challenging topic. As established above, cleanup is performed by reclaiming fragments that are no longer referenced. This is primarily relevant for systems with deduplication enabled. Without deduplication, GC is trivial and can be implemented via a background scan of tuples in cold storage, processed tuple by tuple.

Fragments are eligible for deletion when no references to them remain in the *Links* collection. Determining this state directly is computationally expensive:

- **Maintaining a reverse index** (`fragment.id` $\rightarrow$ `record.id, fragment_index`): While functional, this places a massive burden on the database. It would require $O(1)$ overhead relative to the size of *Links*, which is our largest collection. However, this might be feasible for relatively small-scale installations.

- **Full scan of** *Links*: Heavy and unreliable; shard availability during a full database scan is not guaranteed. This is also only viable for small installations.

- **Reference counting and Access Time tracking**: A more robust approach where refcount ≤ 0 combined with a long period of inactivity strongly suggests a fragment is orphaned. However, this requires frequent metadata updates (though this can be optimized—e.g., updating the "last read" time only once every 24 hours).

Each of the aforementioned variants has significant drawbacks. It can be argued that the first two are strictly unsuitable for "exascale" deployments. However, for massive datasets, a typical "exascale" approach involving offline processing can be applied. This involves emitting events for record creation and fragment-to-record "linking" to an external analytical store. Possible targets include ClickHouse or storage used by Spark/Flink (in practice, events would be streamed into Kafka or RedPanda and then ingested into ClickHouse).

For instance, in ClickHouse, record creation can be represented as `(datetime, record.id, 1)` and deletion as `(datetime, record.id, 0)`, with `record.id` serving as the primary key. By utilizing the `CollapsingMergeTree` engine, the first entry is effectively replaced by the second once the latter is added.

We can then perform a `LEFT OUTER JOIN` with the table representing *Links* on `record.id` where `flag = 1`. If a record has been deleted, its `record.id` will be `NULL` for the corresponding entries in *Links*. This allows us to identify fragments eligible for deletion. Specifically, a fragment "downgrade" approach is proposed:

1. A newly created fragment starts in the `LIVE` state.

2. A `LIVE` fragment identified for deletion transitions to the `DYING` state. This state is marked by the addition of a `death_time` field.

3. A `DYING` fragment identified for deletion is finally purged if the current time exceeds the `death_time`.

However, this necessitates restricting "old" fragments from being reused for deduplication. For example, if a fragment matching a hash is older than 3 months, a new fragment should be created, even if it results in a duplicate.

This scheme is fully viable provided that data is sent both to the metadata database and to the ClickHouse "window" during write operations. For example, after writing the record and its fragment links to the metadata DB, the process returns, but the session remains active to asynchronously add the same data to ClickHouse. If the write fails, the session is retried until the data is successfully ingested.

3.8.3 Compaction and GC Collisions

The previously described logic did not account for potential race conditions. For instance, two concurrent compaction procedures might select the same container and relocate its "live" content into different target containers. This results in an **orphaned fragment**:

1. Compaction #1 updates the reference for fragment *fid*, moving it from the current stripe *A* to a newly created stripe *B*.

2. Compaction #2 performs the same operation, changing the reference from *A* to stripe *C*.

As a result, the system loses track of the fact that stripe *B* contains fragment *fid*, even though the space it occupies is marked as used. This leads to a **space leak**.

Similarly, a collision can occur when compaction and GC results overlap. A fragment *fid* might be relocated to another stripe while a GC process simultaneously attempts to delete it. The sequence of events could unfold as follows:

1. The GC process identifies which container holds the fragment.

2. The GC process updates (decrements) the free space counter for that container.

3. Compaction updates the fragment's reference to point to a new stripe.

4. The GC process deletes the fragment entry from the *Fragments* collection.

Consequently, the container holding the repacked data will store inaccurate information regarding its free space.

While these issues could be addressed using transactions, the overhead might be prohibitive. Instead, all such collisions can be avoided by ensuring that no two compaction procedures overlap, and that no compaction process runs concurrently with the application of GC results. I propose the following approach:

- A **task queue** is established to hold compaction and deletion tasks. A message broker is the simplest implementation for this.

- A **Compaction Scheduler** service is created to enqueue compaction tasks.

- A **GC Adapter** service is created to receive data from the GC process and prepare deletion tasks.

Upon receiving a task, a worker first attempts to acquire a *lease*. The lease is a record in TiKV structured as $high \rightarrow (low, high)$. This acquisition is performed within a transaction:

1. Search for a $lease'$ with the minimum $high' \geq high$. If $low' \leq low$, a collision is detected. The task is rescheduled for a later retry, and no lease is created.

2. Otherwise, the lease is created, and task execution begins.

3. **No TTLs are used.** Using a TTL could lead to a situation where data has been migrated, but metadata updates are only partially complete. Allowing a new process to work on unfinished data could create orphaned resources—occupied containers that are neither referenced nor marked as free. Since operations are performed under a **WAL**, the system will eventually replay the changes to completion and release the lease upon finalization.

Compaction task scheduling: The service iterates through storage regions, partitioning each into segments of random lengths within a specified range. If the last segment of a region is smaller than the minimum allowed size, it encompasses a sub-range of the subsequent region. Crucially, only sub-ranges with indices greater than the current maximum are considered, as regions may change "under the hood" during the process. The set of regions is treated as a circular buffer, where the "next" region for the last one is the first region in TiKV.

GC Adapter logic: This service reads the output from ClickHouse to retrieve fragment identifiers for "downgrading". It identifies the regions containing the relevant containers and groups fragments by region. Regions are further divided into uniform sub-ranges to balance the number of tasks against the *lease* range size: more tasks necessitate smaller ranges to minimize lease duration and task weight. For optimal efficiency, the ClickHouse output should be sorted by fragment index in ascending order.

4 System Invariants

This section defines the properties that must remain true under all operational scenarios, including failures, WAL session retries, and migrations. States that exist only within the scope of specific execution algorithms (e.g., *Hot* $\rightarrow$ *Migrating* $\rightarrow$ *Cold* $\rightarrow$ *Compacted* for a container) are intentionally excluded. These transient states are not part of the system's core data model and may evolve with the implementation. The invariants focus solely on the verifiable properties of the persistent state, which are sufficient to guarantee data integrity and availability.

4.1 Referential Integrity Invariants

I1. Single Source of Truth for Placement. The physical location of a fragment is determined exclusively via the following chain:

$$\mathit{Fragments}[\mathit{fragment.id}].\mathit{container.id} \rightarrow \mathit{Containers}[\mathit{container.id}].\mathit{ref}$$

No other collections contain authoritative information regarding physical placement.

I2. Deterministic Container Placement. For any *container.id*, the *ref* value must belong to exactly one of the following states:

$$\mathit{ref} \in \{\mathit{Hot}(\mathit{hot.id}, \mathit{block.index}),\ \mathit{Tuple}(\mathit{tuple.id}, \mathit{stripe.index})$$

Intermediate or dual states in the metadata are prohibited. The *ref* must point to a tuple and stripe that exist within the *Tuples* collection and the allocator's bitmap.

I3. Single-Fragment Record Integrity. The composition of a *record.id* in *single* format is defined solely by the fragment identifier stored in *record.single*(*fragment_id*).

I4. Multi-Fragment Record Integrity. The composition of a *record.id* in *multi* format is defined exclusively by the following mapping in the *Links* collection:

$$(\mathit{record.id}, \mathit{fragment.index}) \rightarrow \mathit{fragment.id}$$

The *Records* collection does not contain authoritative information regarding the composition of records in *multi* format.

I5. Fragment Immutability. After a record is committed, the following fields:

$$(\mathit{offset},\ \mathit{size},\ \mathit{crc}64,\ \mathit{compression},\ \mathit{blake}3)$$

must remain immutable.любое Any change to the content necessitates the creation of a new *fragment.id*.

I6. Tuple Integrity. For any *tuple.id* in *Tuples*, the number of disks in the *disks* array and their respective roles must strictly conform to the scheme defined in *type*.

4.2 Hot → Cold Migration Invariants

M1. Migration Completion via Reference Swap. A container migration is considered complete only after an atomic replacement:

$$Hot(\cdot) \rightarrow Tuple(\cdot)$$

within *Containers.ref*.

M2. Stripe Commitment Condition. A stripe is considered occupied (*Committed*) only after:

1. Successful write operations for all blocks.
2. Successful atomic update of *Containers.ref*.

M3. Stripe Write Exclusivity. Writing to (*tuple.id*, *stripe.index*) is permitted only under an active Lease issued by the RAFT allocator. A lease for (*tuple.id*, *stripe.index*) is granted only if the disk configuration in *Tuples* allows for the write operation (e.g., all required disks are healthy and accessible).

M4. Migration Idempotency. If *Containers.ref* already points to a $Tuple(\cdot)$, any migration retry must complete without side effects.

M5. Reference Swap Constraints. An atomic transition is only possible as $ref_expected \rightarrow Tuple(\cdot)$, where *ref_expected* is the exact current value of *Containers.ref* (must be a *Hot* or composite *Hot* state, i.e., not *Cold*).

4.3 WAL and Retry Invariants

W1. WAL Enforcement for Multi-Step Operations. Any operation spanning more than one domain (Heat, Cold, or Metadata) must be executed within a WAL session.

W2. Duplicate-Free Retries. The retry of a WAL session must not create additional references, stripes, or containers. A retry operation must either:

- Overwrite the existing values with the same data,
- Or detect a finalized state and terminate execution.

4.4 Deduplication and GC Invariants

D1. Dedup only affects fragment.id selection. Deduplication does not alter the reference model. Connections between a record and its fragments are always expressed via the *Links* collection.

G1. Fragment deletion is only permitted in the absence of references. A fragment may be deleted if and only if there are no rows in *Links* pointing to the given *fragment.id*.

4.5 Execution Invariants

E1. Writing to stripe blocks and buffers is only possible with an active Lease. Any write operation to a cold stripe or a hot buffer is permissible only if a valid *lease.id* exists, issued by the corresponding coordinator (RAFT allocator or hot node). If the Lease is lost or expired, write operations must become impossible.

4.X. Ownership and Concurrent Execution Invariants

O1. Single range owner. In any given moment, there is no more than one process with the right to modify containers within range R. This right is determined by a valid *lease.id* issued by the Range Manager.

O2. Container modification only via range queue. Any operation that changes the state of a container (including fragment redistribution, updates to free space metrics, compaction results, or GC) is performed exclusively as a task in the queue of the corresponding range by its owner.

Q1. Linearizability of range task execution. Range tasks are executed sequentially by a single executor. Each task has a unique identifier $(rangeID, taskID)$. Re-submission or re-execution of a task does not violate correctness (ensured via idempotency or duplicate execution detection).

Q2. Monotonicity of progress. For each range, there is a monotonic progress point *ack*. The *ack* value can only move forward. Recovery after a restart is performed by resuming from the last recorded *ack*.

G2. GC injection relevance. When applying an InjectGC$(container, F)$ task, for each $f \in F$, the action is permissible only if, at the time of execution:

$$\text{Fragments}[f].container_id = container.$$

Otherwise, the operation is a *no-op*.

G3. No direct GC writes to containers. The GC process does not perform direct modifications to the container structure. It only generates range tasks, which are then executed by their respective owners.

M6. Placement persistence before resource commit. Any commit of a cold-layer resource is permissible only after a successful atomic transition:

$$ref_{expected} \rightarrow Tuple(\cdot),$$

executed within a transaction. If the transition fails, the resource is not committed.

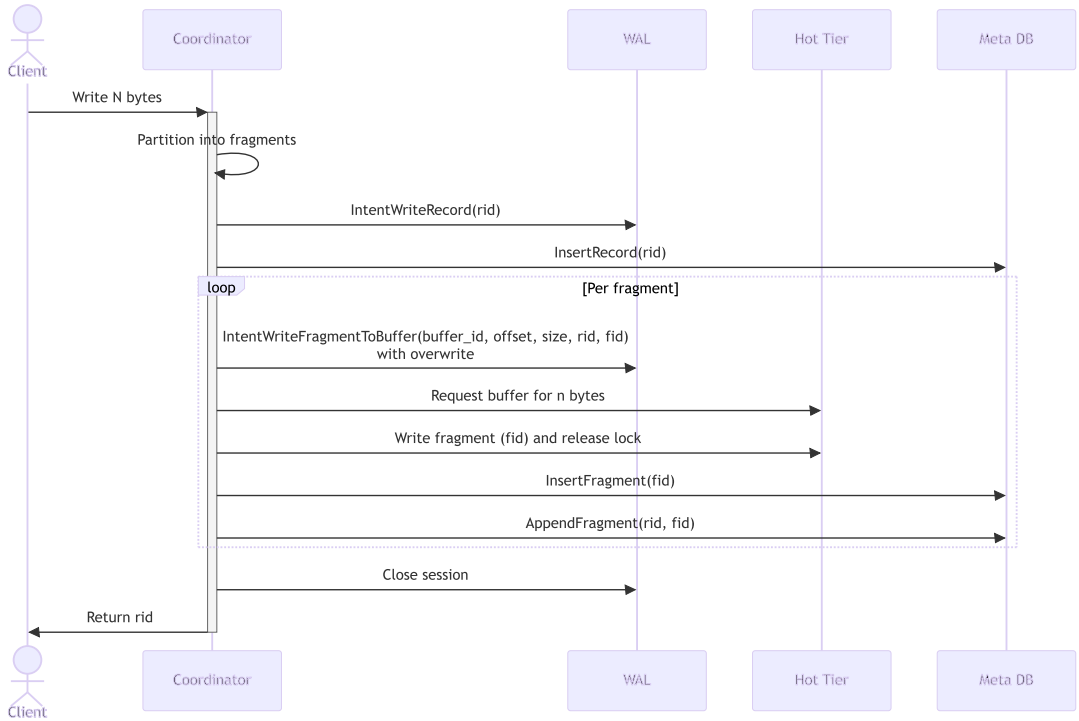
ID1. Canonical key identifier. In all structures requiring uniform distribution, the following key is used:

$$id_{key} = \mathrm{SPECK}_{128}(id_{raw}),$$

where SPECK_{128} is a bijective permutation. This ensures a statistically uniform distribution of containers across ranges.

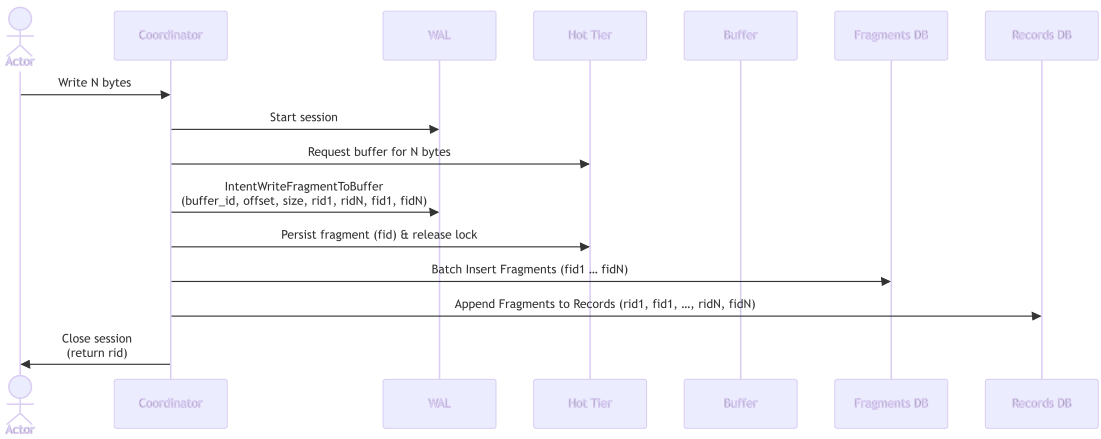
5 Supplementary Materials

5.1 Complete WritePath for multiple fragments



The advantage here is that database operations do not need to be transactional; invariants are ensured by the WAL and the idempotency of the logic.

5.2 Complete MultiWrite



Inserting multiple fragments simultaneously. This process is critical for system performance when inserting small data chunks. Such operations place a high load on the meta-layer due to the massive influx of records. The scheme described here reduces the load on the WAL by using ranges (rid_1, rid_N) and (fid_1, fid_N) to store intents. It also alleviates pressure on the metadata storage through the use of batch writes.

Please note that "original" identifiers are used for $rid_1 \ldots rid_N$ and $fid_1 \ldots fid_N$ instead of their SPECK_{128} values. This is done specifically to enable range notation rather than exhaustive enumeration. In the event of session errors, the recovery process will, of course, operate with transformed values. That is, when verifying rid_K from the $[rid_1 \ldots rid_N]$ range, we are

actually checking the record with the identifier $\text{SPECK}_{128}(rid_K)$.

5.3 On WAL implementation using a message broker and KV storage

A distributed WAL is implemented, maintaining one log per session. The following operations are available within a session:

- Add data – Append.
- Overwrite data – Overwrite.
- Close session.
- Retry session with a delay of at least T seconds.

It is proposed to implement these operations using a message broker and a KV store. NATS + JetStream is suggested as the broker. For the KV store, any persistent solution will work; In-Memory storages like Tarantool or Redis are best suited among persistent options. For disk-based solutions, Aerospike is a strong candidate.

LSM-based storages using RocksDB under the hood are strictly unsuitable. This is because our workload implies:

- Continuous creation of new records with no repeating identifiers.
- Deletion of the vast majority of records within a fraction of a second.
- Most records that survive those initial fractions of a second will still be deleted within seconds.

In these conditions, the skip list in RocksDB (which stores the in-memory LSM table) is highly suboptimal due to its lazy deletion nature. It would lead to extreme IO overhead (write amplification) during subsequent compactions.

5.3.1 Introduction

Upon session initialization, the coordinator receives a fresh session_id (hereafter referred to simply as *id*), which is 16 bytes (128 bits). Under these conditions, we define:

$$K = \text{big_endian}(id)$$

$$K' = \text{append}(K, 0)$$

That is, a zero byte is simply appended to K to form K'.

5.3.2 Working with KV storage

At the start of the operation, a background process is immediately launched to periodically write the current `unixtime` to K' with an established $\text{TTL} = T$. This is done at an interval of, say, one second; higher frequency is unnecessary. The session data is structured as follows:

header	data

The header includes, among other things, the count of retries performed for this session, while the data is a sequence of binary chunks in the following format:

varint(len1)	data1	varint(len2)	data2	...	varint(lenN)	dataN

All session binary data is always held by the coordinator. Under these conditions:

- The **Append** operation corresponds to adding a new `varint(len(data)) + data` to the session data, followed by writing the entire payload (including the header) into the value associated with key K.
- The **Overwrite** operation corresponds to deleting all data following the header, adding a new fragment, and subsequently writing the result.

When a session completes successfully, the value K is deleted from the database. If the session fails, we simply exit. A failure to update K' is considered a session operation failure.

5.3.3 Working with the broker

All coordinators are subscribed to a specific broker topic. Event distribution is not broadcast; instead, messages are delivered to only one of the subscribers (competing consumers pattern).

An event containing only K is sent to this same topic, with an instruction to deliver it no earlier than after T seconds (delayed delivery).

Upon receiving an event, the process follows one of these scenarios:

- There is no record with key K in the database. This indicates the session has been completed. Ack.
- The value in K' was updated too recently (the timestamp is still fresh). Request redelivery after T seconds.
- The retry counter in record K has exceeded the limit. Log the error, delete K, and send an Ack.
- None of the above conditions are met. Initiate session recovery by starting the K' updater, increment the retry counter by one (persisting the new value immediately), and resume operations based on the data recorded in the session. From this point, the process follows the steps described in the "Working with KV" section.

6 Specialized WAL Solution

A Push-Push interaction variant is proposed, operating as follows:

6.1 Consensus Node

The WAL consensus node consists of $2n+1$ machines running MultiRAFT or a similar consensus protocol. It maintains $2n+1$ internal RAFT instances, where each host acts as the leader for one instance and as a follower for the others.

Each host implements an interface of the following type:

```
rpc Session(stream SessionReq) returns (stream SessionResp)

type SessionReq oneof {
  Init(clientClass) // Only the first message in the stream.
  Append(bytes)
  Overwrite(bytes)
  Close             // Only the last message in the stream.
  Repeat(T seconds) // Only the last message in the stream.
  Pulse             // Can occur anywhere in the stream.
}
```

`SessionResp` contains acknowledgments for the performed operations; each response is expected synchronously.

The client connects to a **specific** host of such a node and initializes it using `Init`, then executes a sequence of `Append`/`Overwrite` operations and finishes with a final `Close`/`Repeat`. Meanwhile, a `Pulse` (also acknowledged) is periodically sent in the background.

Active sessions are stored in memory. Sessions scheduled for retry are stored in an LSM-tree, ordered by retry time:

repeat_time	headers	data

If a client disconnects, the session is sent for retry. If a host fails, the former follower that becomes the new session leader immediately sends the active sessions of the inherited RAFT instance for retry. Session retries are initiated via consensus.

6.2 Clients

Services responsible for processing session retries must implement an almost mirror-symmetrical interface:

```
rpc SessionRepeat(stream SessionRepeatReq) returns (stream SessionRepeatResp)
```

Where `SessionRepeatResp` looks almost exactly like `SessionReq`, while `SessionRepeatReq` includes a new branch compared to `SessionResp`:

```
type SessionRepeatReq oneof {
  Deliver(session_headers, session_data) // Only the first in the stream.
  // Further fields are identical to SessionResp.
}
```

During a retry, the node host identifies the class of services required for the session retry and connects to them for execution, starting with the delivery of the session data. This constitutes the second ”Push” in the “Push-Push” scheme.

7 Roadmap

7.1 MVP

1. Implementation of the RAFT allocator.
2. Implementation of a service providing access to machine disks on JBOD using Go and io_uring.
3. Tuples implementation and refinement of the logic for binding disks to a tuple (considering that the host machine may change).
4. Testing block uploads to a ”cold node.”
5. Implementation of the selected hot storage host type using Go + io_uring.
6. Implementation of a hot storage node.
7. WAL scaffolding and integration.
8. Consolidation of metadata logic and final database selection.
9. Implementation of the coordinator.

7.2 GC

Research GC implementations for small-scale installations and for ”exascale” using Clickhouse via Kafka as a sidecar process.

7.3 Logic Evolution

1. Grouping tuples in the allocator by redundancy types. This allows the allocator to consider only those tuples configured for a specific Erasure Coding (EC) or replication scheme.

7.4 Optimization

1. Implementation of a hot storage host with NVMe and NVDIMM using SPDK (selecting either C or Rust).
2. Implementation of JBOD-based nodes in Rust using io_uring.
3. Replacing the placeholder NATS + JetStream WAL with a specialized multi-master Push-Push WAL.
4. Introduction of an archival layer where data and metadata are compressed within a single container, while TiKV stores only Bloom filters and coordinates of the archival layer’s tuples and stripes. Data layout in stripes is identical to the cold layer but utilizes smaller EC schemes (in terms of P) and larger stripe sizes (e.g., 1 GiB instead of 64 MiB). This will alleviate the load on TiKV, which may otherwise struggle with a high volume of small records where metadata weight approaches the weight of the actual data.

5. **High-throughput "Pipes" for small objects.** Introduction of an asynchronous ingestion mechanism for small-scale data. In this mode, data is buffered directly to SSD/NVDIMM without immediate metadata registration. The system provides a post-write callback to bind external identifiers to the resulting `record.id` set, significantly reducing pressure on the metadata layer.

7.5 Description of the "Continuation" Mechanism

During streaming, a buffer may fail. In this case, the system can look for another buffer elsewhere and resume writing not from the beginning, but from the position where the recording on the failed buffer was interrupted. During the transfer process, the first and second buffers are "stitched" together at the break point. This transition is, of course, reflected in the metadata.